

Data Wrangling 🤠

Functions

Lesson 8

Demo: Motivation

```
# conversions.py
temp_1 = 0.0 # in Celsius
temp_2 = 20.0
temp_3 = 37.0

temp_1_f = (temp_1 * 9/5) + 32 # now in Fahrenheit
temp_2_f = (temp_2 * 9/5) + 32
temp_3_f = (temp_3 * 9/5) + 32

print(f"Temperatures in F: {temp_1_f}, {temp_2_f}, {temp_3_f}")
```

Functions

- A **reusable, named block of code** that carries out a specific task
- Takes in **inputs**, runs an **algorithm**, and **returns** a result
- The function only runs when the it is **called**

Defining a Function

```
def celsius_to_fahrenheit(temp):  
    converted_temp = (temp * 9/5) + 32  
    return converted_temp
```

- `def` — keyword that starts a function definition
- `celsius_to_fahrenheit` — the function's name
- `temp` — a **parameter**: a placeholder for input
- `return` — sends the result back to the caller

Function Definition Syntax

```
def [name]([param], ...):  
    [body statements]  
    return [expression]
```

- Function names follow the same rules as variable names
- Body statements must be **indented**
- The `return` statement ends the function and hands back a value

Demo: Defining a Function

```
# conversions.py
def celsius_to_fahrenheit(temp):
    converted_temp = (temp * 9/5) + 32
    return converted_temp

temp_1 = 0.0 # in Celsius
temp_2 = 20.0
temp_3 = 37.0
```

- Defining this function does nothing on its own
- We need to **call** it to use it

Calling a Function

- A **function call** is an expression that runs the function's body
- Arguments are the actual values passed in for each parameter
- Of the form: `[function_name]([arg])`
- The call evaluates to the function's **return value**

```
celsius_to_fahrenheit(0.0)    # returns 32.0
```

```
result = celsius_to_fahrenheit(0.0)  # result is 32.0
```

Functions We've Seen

- You will *not* write every function you use
- We've already seen functions and made function calls
- Recall `print()` and `input()` which take a `str` as an argument:

```
print("Hello, World!")  
input("What is your name: ")
```

- Recall `randint()` which takes two `int`s as a range to pick between:

```
randint(0, 3) # returns a number between 0 and 3
```


Demo: Calling a Function in the REPL

```
>>> from lessons.conversions import celsius_to_fahrenheit
>>> celsius_to_fahrenheit(5)
41.0
>>> celsius_to_fahrenheit(15.5)
59.9
```

Demo: Variable Scope

```
>>> temp_in_f = celsius_to_fahrenheit(50.0)
>>> temp_in_f
122.0
>>> temp
Traceback (most recent call last):
  File "<python-input-21>", line 1, in <module>
    temp
NameError: name 'temp' is not defined
>>> converted_temp
Traceback (most recent call last):
  File "<python-input-22>", line 1, in <module>
    converted_temp
NameError: name 'converted_temp' is not defined
```

Demo: Function Call in Program

```
# conversions.py
def celsius_to_fahrenheit(temp):
    converted_temp = (temp * 9/5) + 32
    return converted_temp

temp_1 = 0.0 # in Celsius
temp_2 = 20.0
temp_3 = 37.0

temp_1_f = celsius_to_fahrenheit(temp_1) # 32.0
temp_2_f = celsius_to_fahrenheit(temp_2) # 68.0
temp_3_f = celsius_to_fahrenheit(temp_3)

print(f"Temperatures in C: {temp_1}, {temp_2}, {temp_3}")
print(f"Temperatures in F: {temp_1_f}, {temp_2_f}, {temp_3_f}")
```

Multiple Parameters

- Functions can take more than one input — separate **parameters** and **arguments** with commas

```
def power(base, exp):  
    return base ** exp
```

- Arguments are assigned to parameters **in order**

```
>>> power(3, 2)  
9  
>>> power(2, 3)  
8
```

Return Statements

- Once a `return` is hit, **no further lines in the function run**
- A function can have multiple `return` statements – only one will run

```
def larger(x, y):  
    if x > y:  
        return x  
    return y
```

```
>>> larger(10, 5)  
10  
>>> larger(3, 7)  
7
```

Why Functions?

- Functions helps us **reuse**, **abstract**, and **organize** our code
- Once you've defined a function, you can *use* it without worrying about its steps